

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: Max Epifanio Varela Intriago

PARALELO: A

SEMESTRE: Quinto

ACTIVIDAD: Actividad #8: Desarrollo del Backend y Base de Datos (API REST)

PERIODO SEPTIEMBRE 2025 - ENERO 2026



Actividades a desarrollar en la Unidad 4

Actividad #8: Desarrollo del Backend y Base de Datos (API REST)

1. Objetivo

Construir la arquitectura del lado del servidor (Backend) y la base de datos del Sistema de Gestión de Incidentes (Help Desk), aplicando Node.js, Express y MongoDB para exponer una API RESTful que permita realizar las operaciones CRUD sobre los tickets reportados.

2. Configuración de la base de datos

Se diseñó la colección "tickets" en MongoDB (mediante Mongoose) con los siguientes atributos: titulo, descripcion, solicitante, categoria (Red, Hardware, Software), prioridad (Alta, Media, Baja) y estado (Abierto, En Progreso, Cerrado). El identificador (_id) lo genera MongoDB automáticamente.

Para el entorno de desarrollo local se utilizó una instancia de MongoDB en memoria (mongodb-memory-server), que simula un servidor MongoDB real sin necesidad de instalación previa. Para producción, el proyecto está preparado para conectarse a MongoDB Atlas mediante la variable de entorno MONGODB_URI (ver backend/.env.example).

3. Arquitectura del servidor

El backend se desarrolló con Node.js y el framework Express, siguiendo una estructura en capas que separa responsabilidades:

- config/db.js: gestiona la conexión a MongoDB.
- models/Ticket.js: define el esquema de datos (Mongoose).
- controllers/ticketController.js: contiene la lógica de negocio del CRUD.
- routes/tickets.js: define los endpoints HTTP y los enlaza con el controlador.
- server.js: punto de entrada de la aplicación Express.

4. Endpoints de la API REST desarrollados

GET	/api/tickets	-> Lista todos los tickets registrados
GET	/api/tickets/:id	-> Obtiene un ticket específico por su id
POST	/api/tickets	-> Registra un nuevo ticket (incidente)
PUT	/api/tickets/:id	-> Actualiza el estado o los detalles de un ticket
DELETE	/api/tickets/:id	-> Elimina un ticket

5. Código fuente principal del controlador (ticketController.js)

```
const Ticket = require('../models/Ticket');

// GET /api/tickets - listar todos los incidentes
```



```
exports.listarTickets = async (req, res) => {
  try {
    const tickets = await Ticket.find().sort({ createdAt: -1 });
    res.json(tickets);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};

// GET /api/tickets/:id - buscar un ticket específico
exports.obtenerTicket = async (req, res) => {
  try {
    const ticket = await Ticket.findById(req.params.id);
    if (!ticket) return res.status(404).json({ error: 'Ticket no encontrado' });
    res.json(ticket);
  } catch (err) {
    res.status(400).json({ error: 'ID de ticket inválido' });
  }
};

// POST /api/tickets - registrar un nuevo incidente
exports.crearTicket = async (req, res) => {
  try {
    const { titulo, descripcion, solicitante, categoria, prioridad } = req.body;
    const ticket = await Ticket.create({ titulo, descripcion, solicitante, categoria,
prioridad });
    res.status(201).json(ticket);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
};

// PUT /api/tickets/:id - actualizar estado o detalles de un ticket
exports.actualizarTicket = async (req, res) => {
  try {
```



```
const ticket = await Ticket.findByIdAndUpdate(req.params.id, req.body, {
  new: true,
  runValidators: true,
});
if (!ticket) return res.status(404).json({ error: 'Ticket no encontrado' });
res.json(ticket);
} catch (err) {
  res.status(400).json({ error: err.message });
}
};

// DELETE /api/tickets/:id - eliminar un registro
exports.eliminarTicket = async (req, res) => {
  try {
    const ticket = await Ticket.findByIdAndDelete(req.params.id);
    if (!ticket) return res.status(404).json({ error: 'Ticket no encontrado' });
    res.json({ mensaje: 'Ticket eliminado correctamente' });
  } catch (err) {
    res.status(400).json({ error: 'ID de ticket invalido' });
  }
};
```

6. Pruebas de conectividad realizadas

Se probaron los cinco endpoints con cURL, confirmando respuestas exitosas (200/201) en formato JSON, así como el caso de error 404 al buscar/eliminar un ticket inexistente. Las capturas de pantalla de estas pruebas se incluyen en la sección de Evidencias.

7. Control de versiones

El código del backend fue subido al repositorio de GitHub en la rama feature/backend-api y posteriormente fusionado a la rama develop, tal como lo solicita la guía de la actividad.

Repositorio: <https://github.com/MaxVarela240/SISTEMA-HELP-DESK>



Evidencias

Repositorio de GitHub: <https://github.com/MaxVarela240/SISTEMA-HELP-DESK> (rama *feature/backend-api*, fusionada a *develop*).

El código fuente principal del controlador (ticketController.js) se incluyó en la sección de Desarrollo de esta actividad.

A continuación, las capturas de las pruebas realizadas a los 5 endpoints de la API (GET, GET/:id, POST, PUT, DELETE), obtenidas ejecutando las peticiones reales contra el servidor local (<http://localhost:4000>) y verificando los códigos de estado HTTP 200/201/404 junto con las respuestas JSON:

Evidencia de pruebas de la API REST - Sistema Help Desk (parte 1/2)

Ejecutado con cURL/fetch contra <http://localhost:4000> - 08/08/2026

GET

/

HTTP 200

```
{
  "mensaje": "API Help Desk activa"
}
```

GET

/api/tickets

HTTP 200

```
[]
```

POST

/api/tickets

HTTP 201

```
{
  "titulo": "No enciende el monitor",
  "descripcion": "El monitor de la sala 3 no enciende",
  "solicitante": "Maria Perez",
  "categoria": "Hardware",
  "prioridad": "Alta",
  "estado": "Abierto",
  "_id": "6a77d9d3487103a6cfd0c7a9",
  "createdAt": "2026-08-09T01:37:23.480Z",
  "updatedAt": "2026-08-09T01:37:23.480Z",
  "__v": 0
}
```

GET

/api/tickets/6a77d9d3487103a6cfd0c7a9

HTTP 200

```
{
  "_id": "6a77d9d3487103a6cfd0c7a9",
  "titulo": "No enciende el monitor",
  "descripcion": "El monitor de la sala 3 no enciende",
  "solicitante": "Maria Perez",
  "categoria": "Hardware",
  "prioridad": "Alta",
  "estado": "Abierto",
  "createdAt": "2026-08-09T01:37:23.480Z",
  "updatedAt": "2026-08-09T01:37:23.480Z",
  "__v": 0
}
```



Evidencia de pruebas de la API REST - Sistema Help Desk (parte 2/2)

Ejecutado con cURL/fetch contra http://localhost:4000 - 08/08/2026

PUT /api/tickets/6a77d9d3487103a6cfd0c7a9

HTTP 200

```
{
  "_id": "6a77d9d3487103a6cfd0c7a9",
  "titulo": "No enciende el monitor",
  "descripcion": "El monitor de la sala 3 no enciende",
  "solicitante": "Maria Perez",
  "categoria": "Hardware",
  "prioridad": "Alta",
  "estado": "En Progreso",
  "createdAt": "2026-08-09T01:37:23.480Z",
  "updatedAt": "2026-08-09T01:37:23.536Z",
  "__v": 0
}
```

GET /api/tickets

HTTP 200

```
[
  {
    "_id": "6a77d9d3487103a6cfd0c7a9",
    "titulo": "No enciende el monitor",
    "descripcion": "El monitor de la sala 3 no enciende",
    "solicitante": "Maria Perez",
    "categoria": "Hardware",
    "prioridad": "Alta",
    "estado": "En Progreso",
    "createdAt": "2026-08-09T01:37:23.480Z",
    "updatedAt": "2026-08-09T01:37:23.536Z",
    "__v": 0
  }
]
```

DELETE /api/tickets/6a77d9d3487103a6cfd0c7a9

HTTP 200

```
{
  "mensaje": "Ticket eliminado correctamente"
}
```

GET /api/tickets/6a77d9d3487103a6cfd0c7a9

HTTP 404

```
{
  "error": "Ticket no encontrado"
}
```

Bibliografía

Building REST services with Spring. (2021). Spring.io.
<https://spring.io/guides/tutorials/rest/>

Express.js. (2024). Express - Node.js web application framework.
<https://expressjs.com/>

MongoDB, Inc. (2024). MongoDB Manual. <https://docs.mongodb.com/manual/>

Mongoose. (2024). Mongoose ODM v8. <https://mongoosejs.com/docs/>

OpenJS Foundation. (2024). Node.js Documentation. <https://nodejs.org/en/docs>

Universidad Técnica de Manabí. (2026). Compendio DSI Unidad 4: Desarrollo de sistemas y bases de datos. UTM Online.